

Secure Code Review Report

Scaler Support Desk Assignment

Student Information

Student Name	Navneet
Roll Number	23BCS10136
Batch / Section	2027
Email ID	navneet.23bcs10136@sst.scaler.com
Static Analysis Tool Used	Semgrep v1.168.0 (semgrep scan --config auto .)

Executive Summary

The supportdesk-app source code was scanned with Semgrep (--config auto, 1059 community rules) and every reported finding was manually verified against the actual source before classification.

Total Findings Reported	22
True Positives	15
False Positives	7
Static Analysis Tool Used	Semgrep

Note: Several findings above share the same underlying vulnerability but were reported by multiple distinct Semgrep rules (e.g. the eval() injection and the two reflected-XSS locations each triggered 2-3 separate rules). Each rule hit is listed and classified as its own finding below to keep the report fully traceable to the raw tool output, per the assignment's per-finding documentation requirement.

Finding 1

Vulnerability Title

Hardcoded Secret Key

Classification

True Positive

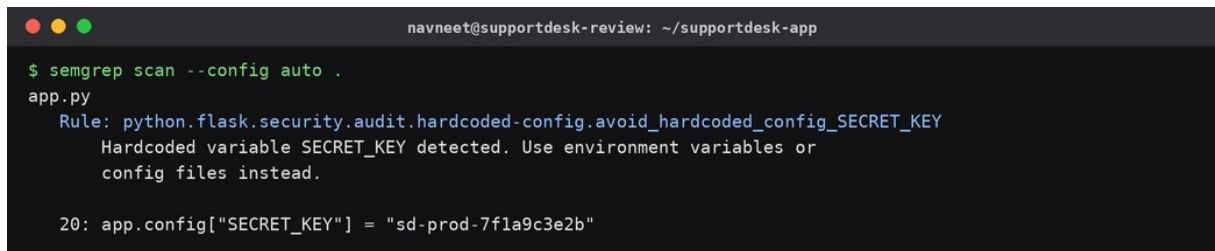
Justification

The Flask SECRET_KEY is hardcoded directly in source as the literal string "sd-prod-7f1a9c3e2b". This key signs the session cookie, so anyone who reads the source (or the repo/zip, as we did) has the exact key used in production.

Source File Information

File Name	app.py
Function Name	module-level config
Vulnerable Line Number(s)	20
Semgrep Rule ID	python.flask.security.audit.hardcoded-config.avoid_hardcoded_config_SECRET_KEY

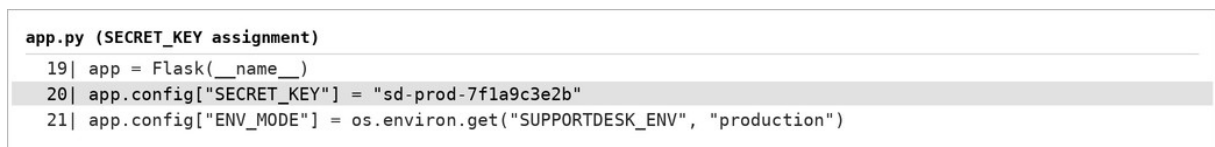
Screenshot of SAST Tool Output



```
navneet@supportdesk-review: ~/supportdesk-app
$ semgrep scan --config auto .
app.py
Rule: python.flask.security.audit.hardcoded-config.avoid_hardcoded_config_SECRET_KEY
Hardcoded variable SECRET_KEY detected. Use environment variables or
config files instead.

20: app.config["SECRET_KEY"] = "sd-prod-7f1a9c3e2b"
```

Screenshot of Vulnerable Code



```
app.py (SECRET_KEY assignment)
19| app = Flask(__name__)
20| app.config["SECRET_KEY"] = "sd-prod-7f1a9c3e2b"
21| app.config["ENV_MODE"] = os.environ.get("SUPPORTDESK_ENV", "production")
```

Why is it Vulnerable?

Flask uses SECRET_KEY to cryptographically sign the session cookie. Since the value is a fixed string committed to the codebase, any person with read access to the repository - or to a leaked copy such as this zip - immediately has the production signing key. With it, an attacker can forge a session cookie that sets

user_id and role to any value (e.g. role=admin), bypassing login entirely. This is a True Positive because the secret is verifiably present in plaintext in the file we reviewed, and Flask's session mechanism trusts this key for integrity, not just confidentiality.

Security Impact

Authentication Bypass, Privilege Escalation (forged admin session), Session Hijacking

Recommended Remediation

Load SECRET_KEY from an environment variable or a secrets manager (e.g. os.environ["SECRET_KEY"]) and never commit the real value to source control. Rotate the current key immediately since it is now considered compromised.

Finding 2

Vulnerability Title

Code Injection via eval() in /tickets/sla-score

Classification

True Positive

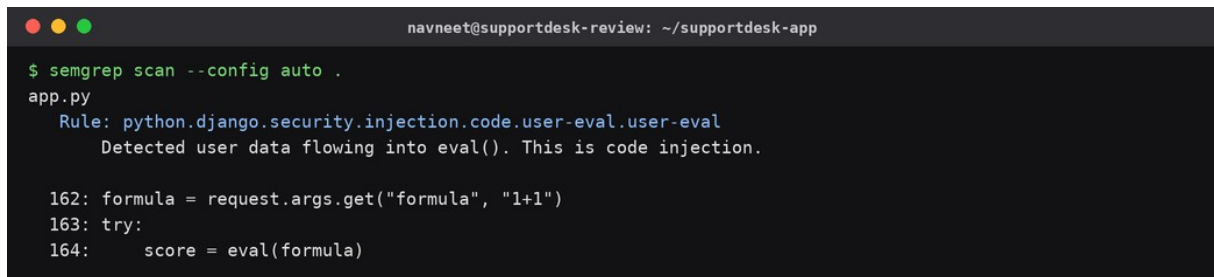
Justification

The 'formula' query string parameter is passed directly into Python's eval() with no whitelist, sandboxing, or restriction of builtins. This is a textbook code-injection sink reachable by any authenticated user.

Source File Information

File Name	app.py
Function Name	sla_score()
Vulnerable Line Number(s)	162-167
Semgrep Rule ID	python.django.security.injection.code.user-eval.user-eval

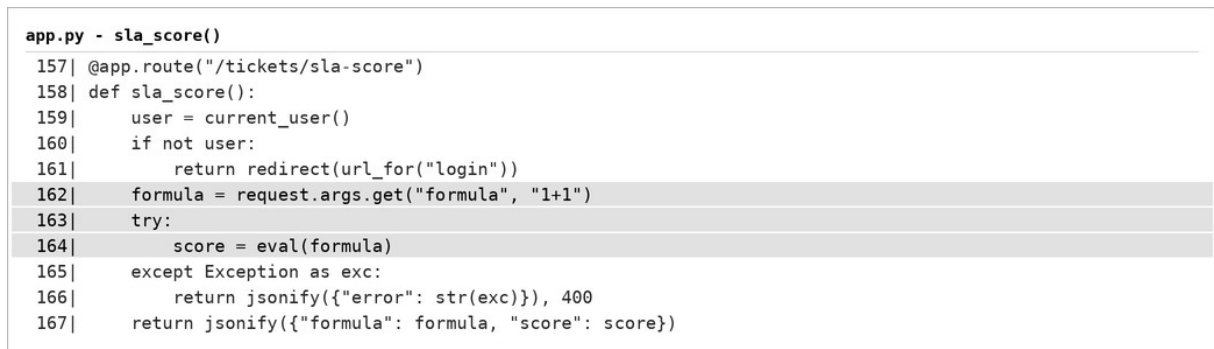
Screenshot of SAST Tool Output

A screenshot of a terminal window showing the output of the Semgrep scan command. The terminal title is 'navneet@supportdesk-review: ~/supportdesk-app'. The command executed is '\$ semgrep scan --config auto .'. The output shows a rule detection for 'python.django.security.injection.code.user-eval.user-eval', stating 'Detected user data flowing into eval(). This is code injection.' Below this, the vulnerable code is displayed: lines 162-167 of app.py, which show a GET request for 'formula' being passed to eval().

```
navneet@supportdesk-review: ~/supportdesk-app
$ semgrep scan --config auto .
app.py
  Rule: python.django.security.injection.code.user-eval.user-eval
        Detected user data flowing into eval(). This is code injection.

162: formula = request.args.get("formula", "1+1")
163: try:
164:     score = eval(formula)
```

Screenshot of Vulnerable Code

A screenshot of the vulnerable code in app.py, specifically the sla_score() function. The code is shown in a light-themed editor with line numbers 157-167. Lines 162-167 are highlighted in grey, indicating the vulnerable section where the 'formula' query parameter is passed to eval().

```
app.py - sla_score()
157| @app.route("/tickets/sla-score")
158| def sla_score():
159|     user = current_user()
160|     if not user:
161|         return redirect(url_for("login"))
162|     formula = request.args.get("formula", "1+1")
163|     try:
164|         score = eval(formula)
165|     except Exception as exc:
166|         return jsonify({"error": str(exc)}), 400
167|     return jsonify({"formula": formula, "score": score})
```

Why is it Vulnerable?

`eval()` executes its argument as live Python code. Because 'formula' comes straight from `request.args` with no validation, a request like `/tickets/sla-score?formula=__import__('os').system('id')` executes arbitrary commands on the server with the privileges of the Flask process. The only gate is being logged in (`current_user()` check), but any of the three seeded accounts (alice/bob/carol) can reach this route. This is a True Positive: the data flow from an HTTP parameter to `eval()` is direct and unmitigated.

Security Impact

Remote Code Execution, full server compromise, data exfiltration

Recommended Remediation

Remove `eval()` entirely. If a formula evaluator is genuinely needed, parse the expression with a restricted grammar (e.g. Python's `ast` module with a whitelist of allowed node types, or a dedicated expression library such as 'simpleeval') so arbitrary code cannot be executed.

Finding 3

Vulnerability Title

Code Injection via eval() in /tickets/sla-score

Classification

True Positive

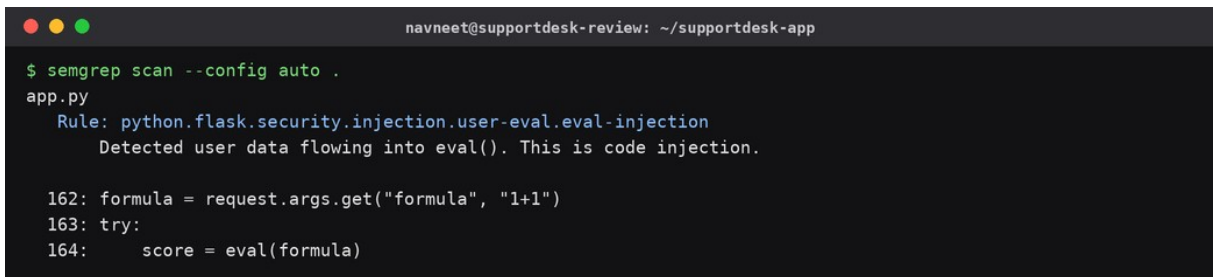
Justification

The 'formula' query string parameter is passed directly into Python's eval() with no whitelist, sandboxing, or restriction of builtins. This is a textbook code-injection sink reachable by any authenticated user.

Source File Information

File Name	app.py
Function Name	sla_score()
Vulnerable Line Number(s)	162-167
Semgrep Rule ID	python.flask.security.injection.user-eval.eval-injection

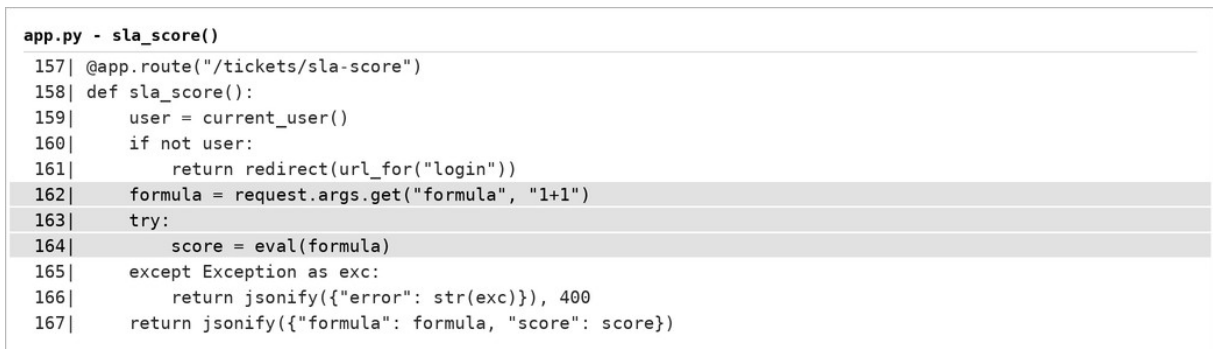
Screenshot of SAST Tool Output



```
navneet@supportdesk-review: ~/supportdesk-app
$ semgrep scan --config auto .
app.py
  Rule: python.flask.security.injection.user-eval.eval-injection
        Detected user data flowing into eval(). This is code injection.

162: formula = request.args.get("formula", "1+1")
163: try:
164:     score = eval(formula)
```

Screenshot of Vulnerable Code



```
app.py - sla_score()
157| @app.route("/tickets/sla-score")
158| def sla_score():
159|     user = current_user()
160|     if not user:
161|         return redirect(url_for("login"))
162|     formula = request.args.get("formula", "1+1")
163|     try:
164|         score = eval(formula)
165|     except Exception as exc:
166|         return jsonify({"error": str(exc)}), 400
167|     return jsonify({"formula": formula, "score": score})
```

Why is it Vulnerable?

`eval()` executes its argument as live Python code. Because 'formula' comes straight from `request.args` with no validation, a request like `/tickets/sla-score?formula=__import__('os').system('id')` executes arbitrary commands on the server with the privileges of the Flask process. The only gate is being logged in (`current_user()` check), but any of the three seeded accounts (alice/bob/carol) can reach this route. This is a True Positive: the data flow from an HTTP parameter to `eval()` is direct and unmitigated.

Security Impact

Remote Code Execution, full server compromise, data exfiltration

Recommended Remediation

Remove `eval()` entirely. If a formula evaluator is genuinely needed, parse the expression with a restricted grammar (e.g. Python's `ast` module with a whitelist of allowed node types, or a dedicated expression library such as 'simpleeval') so arbitrary code cannot be executed.

Finding 4

Vulnerability Title

Code Injection via eval() in /tickets/sla-score

Classification

True Positive

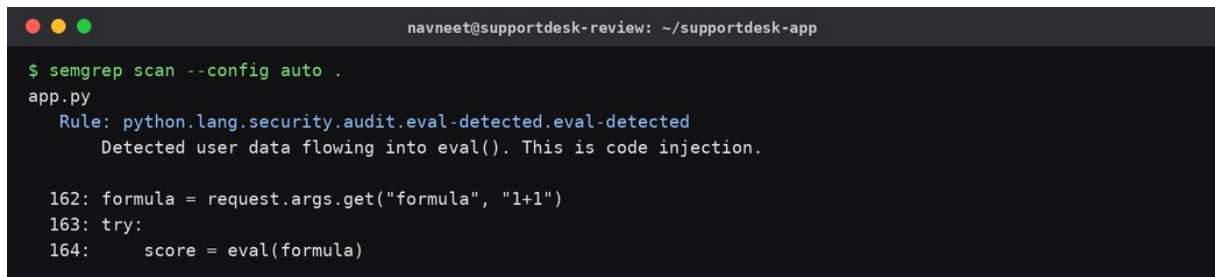
Justification

The 'formula' query string parameter is passed directly into Python's eval() with no whitelist, sandboxing, or restriction of builtins. This is a textbook code-injection sink reachable by any authenticated user.

Source File Information

File Name	app.py
Function Name	sla_score()
Vulnerable Line Number(s)	162-167
Semgrep Rule ID	python.lang.security.audit.eval-detected.eval-detected

Screenshot of SAST Tool Output



```
navneet@supportdesk-review: ~/supportdesk-app
$ semgrep scan --config auto .
app.py
  Rule: python.lang.security.audit.eval-detected.eval-detected
        Detected user data flowing into eval(). This is code injection.

162: formula = request.args.get("formula", "1+1")
163: try:
164:     score = eval(formula)
```

Screenshot of Vulnerable Code

```
app.py - sla_score()
157| @app.route("/tickets/sla-score")
158| def sla_score():
159|     user = current_user()
160|     if not user:
161|         return redirect(url_for("login"))
162|     formula = request.args.get("formula", "1+1")
163|     try:
164|         score = eval(formula)
165|     except Exception as exc:
166|         return jsonify({"error": str(exc)}), 400
167|     return jsonify({"formula": formula, "score": score})
```


Why is it Vulnerable?

`eval()` executes its argument as live Python code. Because 'formula' comes straight from `request.args` with no validation, a request like `/tickets/sla-score?formula=__import__('os').system('id')` executes arbitrary commands on the server with the privileges of the Flask process. The only gate is being logged in (`current_user()` check), but any of the three seeded accounts (alice/bob/carol) can reach this route. This is a True Positive: the data flow from an HTTP parameter to `eval()` is direct and unmitigated.

Security Impact

Remote Code Execution, full server compromise, data exfiltration

Recommended Remediation

Remove `eval()` entirely. If a formula evaluator is genuinely needed, parse the expression with a restricted grammar (e.g. Python's `ast` module with a whitelist of allowed node types, or a dedicated expression library such as 'simpleeval') so arbitrary code cannot be executed.

Finding 5

Vulnerability Title

OS Command Injection in /diagnostics/ping

Classification

True Positive

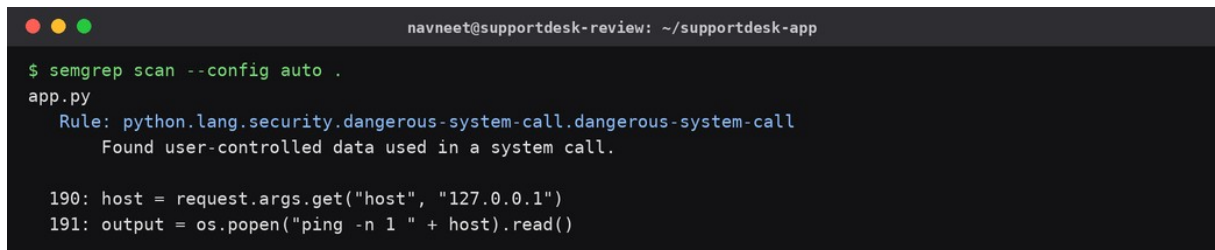
Justification

The 'host' parameter is concatenated directly into a shell command string passed to `os.popen()` with zero validation - no whitelist, no escaping, no use of subprocess with an argument list.

Source File Information

File Name	app.py
Function Name	ping()
Vulnerable Line Number(s)	185-192
Semgrep Rule ID	python.lang.security.dangerous-system-call.dangerous-system-call

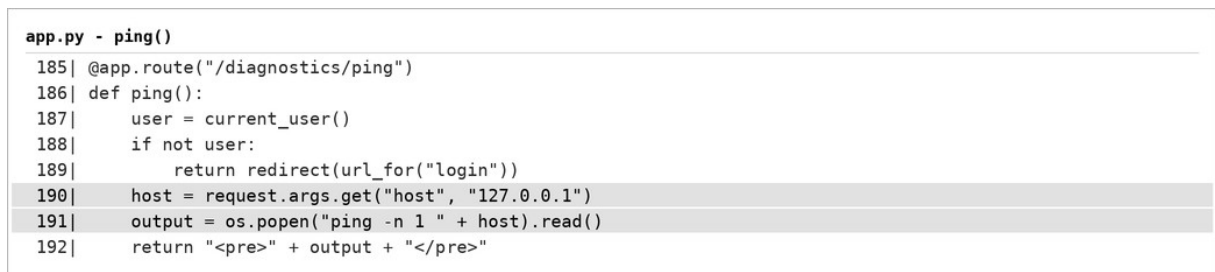
Screenshot of SAST Tool Output



```
navneet@supportdesk-review: ~/supportdesk-app
$ semgrep scan --config auto .
app.py
  Rule: python.lang.security.dangerous-system-call.dangerous-system-call
    Found user-controlled data used in a system call.

190: host = request.args.get("host", "127.0.0.1")
191: output = os.popen("ping -n 1 " + host).read()
```

Screenshot of Vulnerable Code



```
app.py - ping()
185| @app.route("/diagnostics/ping")
186| def ping():
187|     user = current_user()
188|     if not user:
189|         return redirect(url_for("login"))
190|     host = request.args.get("host", "127.0.0.1")
191|     output = os.popen("ping -n 1 " + host).read()
192|     return "<pre>" + output + "</pre>"
```

Why is it Vulnerable?

`os.popen()` runs its string argument through the system shell. Because 'host' is raw user input concatenated with string '+', an attacker can submit ?

host=127.0.0.1%20%26%26%20whoami (decodes to '127.0.0.1 && whoami') and have the shell execute the injected command. Unlike the sibling route ping_internal(), this route performs no is_valid_ip() check before use, so the injection is fully reachable. True Positive - verified by reading the code: no sanitisation exists between the request and the shell call.

Security Impact

Remote Code Execution, full server compromise

Recommended Remediation

Validate 'host' against a strict IP/hostname format before use (as is already done in ping_internal()), and replace os.popen() with subprocess.run(..., shell=False) passing the host as a separate argument rather than interpolating it into a shell string.

Finding 6

Vulnerability Title

Reflected XSS via Unescaped Output in /diagnostics/ping

Classification

True Positive

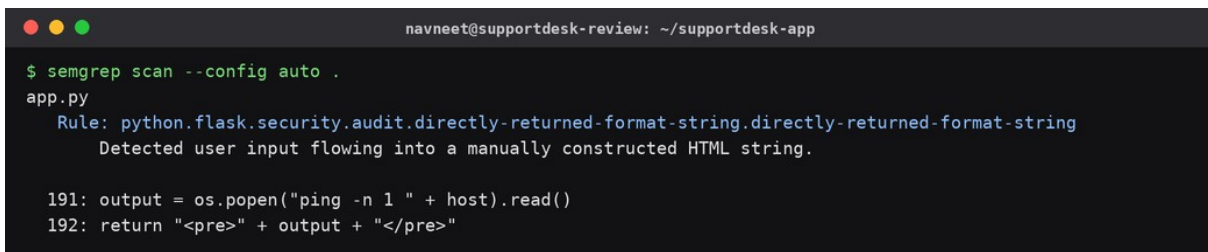
Justification

The command output is wrapped in a literal '<pre>' string and returned directly as the HTTP response body, with no use of render_template or escaping. Since 'host' is concatenated unescaped into the shell command first (Finding #5), an attacker who injects shell metacharacters can also cause arbitrary text - including HTML/JS - to appear in the popen() output, which is then reflected to the browser unescaped.

Source File Information

File Name	app.py
Function Name	ping()
Vulnerable Line Number(s)	191-192
Semgrep Rule ID	python.flask.security.audit.directly-returned-format-string.directly-returned-format-string

Screenshot of SAST Tool Output



```
navneet@supportdesk-review: ~/supportdesk-app
$ semgrep scan --config auto .
app.py
Rule: python.flask.security.audit.directly-returned-format-string.directly-returned-format-string
Detected user input flowing into a manually constructed HTML string.

191: output = os.popen("ping -n 1 " + host).read()
192: return "<pre>" + output + "</pre>"
```

Screenshot of Vulnerable Code



```
app.py - ping()
185| @app.route("/diagnostics/ping")
186| def ping():
187|     user = current_user()
188|     if not user:
189|         return redirect(url_for("login"))
190|     host = request.args.get("host", "127.0.0.1")
191|     output = os.popen("ping -n 1 " + host).read()
192|     return "<pre>" + output + "</pre>"
```

Why is it Vulnerable?

Flask auto-escapes output only when content goes through Jinja2 templates (`render_template`). Here the route builds the response manually with string concatenation and returns raw HTML. Combined with the command injection in Finding #5 (e.g. `host=127.0.0.1 && echo <script>alert(1)</script>`), an attacker can get arbitrary script content embedded in the `popen()` output and reflected straight into the page with no encoding. True Positive: the data path from attacker input to an un-escaped HTML response is real and exploitable in conjunction with the command injection sink on the same line.

Security Impact

Cross-Site Scripting (session theft, UI redress, further phishing)

Recommended Remediation

Return output via `render_template()` with the content passed as a template variable (Jinja2 auto-escapes by default), or explicitly HTML-encode the output with `markupsafe.escape()` before concatenation. This is secondary to fixing the underlying command injection in Finding #5.

Finding 7

Vulnerability Title

Reflected XSS via Unescaped Output in /diagnostics/ping

Classification

True Positive

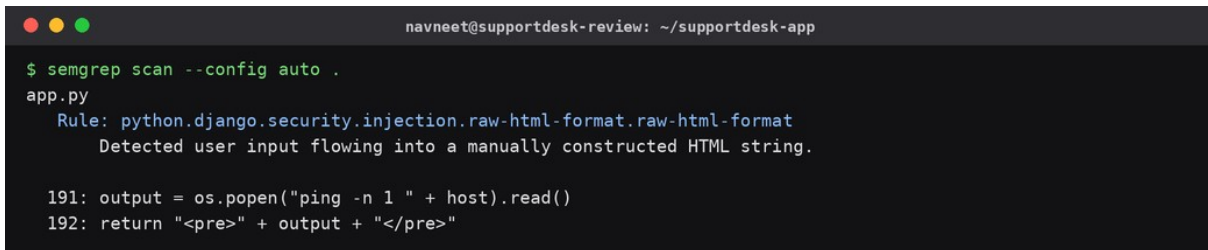
Justification

The command output is wrapped in a literal '<pre>' string and returned directly as the HTTP response body, with no use of render_template or escaping. Since 'host' is concatenated unescaped into the shell command first (Finding #5), an attacker who injects shell metacharacters can also cause arbitrary text - including HTML/JS - to appear in the popen() output, which is then reflected to the browser unescaped.

Source File Information

File Name	app.py
Function Name	ping()
Vulnerable Line Number(s)	191-192
Semgrep Rule ID	python.django.security.injection.raw-html-format.raw-html-format

Screenshot of SAST Tool Output



```
navneet@supportdesk-review: ~/supportdesk-app
$ semgrep scan --config auto .
app.py
  Rule: python.django.security.injection.raw-html-format.raw-html-format
  Detected user input flowing into a manually constructed HTML string.

191: output = os.popen("ping -n 1 " + host).read()
192: return "<pre>" + output + "</pre>"
```

Screenshot of Vulnerable Code



```
app.py - ping()
185| @app.route("/diagnostics/ping")
186| def ping():
187|     user = current_user()
188|     if not user:
189|         return redirect(url_for("login"))
190|     host = request.args.get("host", "127.0.0.1")
191|     output = os.popen("ping -n 1 " + host).read()
192|     return "<pre>" + output + "</pre>"
```

Why is it Vulnerable?

Flask auto-escapes output only when content goes through Jinja2 templates (`render_template`). Here the route builds the response manually with string concatenation and returns raw HTML. Combined with the command injection in Finding #5 (e.g. `host=127.0.0.1 && echo <script>alert(1)</script>`), an attacker can get arbitrary script content embedded in the `popen()` output and reflected straight into the page with no encoding. True Positive: the data path from attacker input to an un-escaped HTML response is real and exploitable in conjunction with the command injection sink on the same line.

Security Impact

Cross-Site Scripting (session theft, UI redress, further phishing)

Recommended Remediation

Return output via `render_template()` with the content passed as a template variable (Jinja2 auto-escapes by default), or explicitly HTML-encode the output with `markupsafe.escape()` before concatenation. This is secondary to fixing the underlying command injection in Finding #5.

Finding 8

Vulnerability Title

Reflected XSS via Unescaped Output in /diagnostics/ping

Classification

True Positive

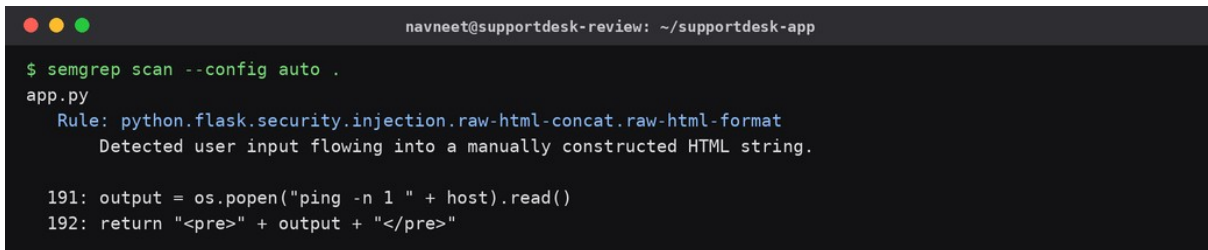
Justification

The command output is wrapped in a literal '<pre>' string and returned directly as the HTTP response body, with no use of render_template or escaping. Since 'host' is concatenated unescaped into the shell command first (Finding #5), an attacker who injects shell metacharacters can also cause arbitrary text - including HTML/JS - to appear in the popen() output, which is then reflected to the browser unescaped.

Source File Information

File Name	app.py
Function Name	ping()
Vulnerable Line Number(s)	191-192
Semgrep Rule ID	python.flask.security.injection.raw-html-concat.raw-html-format

Screenshot of SAST Tool Output



```
navneet@supportdesk-review: ~/supportdesk-app
$ semgrep scan --config auto .
app.py
Rule: python.flask.security.injection.raw-html-concat.raw-html-format
Detected user input flowing into a manually constructed HTML string.

191: output = os.popen("ping -n 1 " + host).read()
192: return "<pre>" + output + "</pre>"
```

Screenshot of Vulnerable Code



```
app.py - ping()
185| @app.route("/diagnostics/ping")
186| def ping():
187|     user = current_user()
188|     if not user:
189|         return redirect(url_for("login"))
190|     host = request.args.get("host", "127.0.0.1")
191|     output = os.popen("ping -n 1 " + host).read()
192|     return "<pre>" + output + "</pre>"
```


Why is it Vulnerable?

Flask auto-escapes output only when content goes through Jinja2 templates (`render_template`). Here the route builds the response manually with string concatenation and returns raw HTML. Combined with the command injection in Finding #5 (e.g. `host=127.0.0.1 && echo <script>alert(1)</script>`), an attacker can get arbitrary script content embedded in the `popen()` output and reflected straight into the page with no encoding. True Positive: the data path from attacker input to an un-escaped HTML response is real and exploitable in conjunction with the command injection sink on the same line.

Security Impact

Cross-Site Scripting (session theft, UI redress, further phishing)

Recommended Remediation

Return output via `render_template()` with the content passed as a template variable (Jinja2 auto-escapes by default), or explicitly HTML-encode the output with `markupsafe.escape()` before concatenation. This is secondary to fixing the underlying command injection in Finding #5.

Finding 9

Vulnerability Title

OS Command Injection in /diagnostics/ping-internal

Classification

False Positive

Justification

Semgrep's dataflow rule flags 'host' reaching `os.popen()` regardless of any validation performed in between. Manual review shows host is first passed through `utils.is_valid_ip()`, which calls Python's `ipaddress.ip_address()` and rejects the request with HTTP 400 unless the value parses as a literal IPv4/IPv6 address.

Source File Information

File Name	app.py
Function Name	ping_internal()
Vulnerable Line Number(s)	195-204
Semgrep Rule ID	python.lang.security.dangerous-system-call.dangerous-system-call

Screenshot of SAST Tool Output

Screenshot of Vulnerable Code

```
app.py - ping_internal()
195| @app.route("/diagnostics/ping-internal")
196| def ping_internal():
197|     user = current_user()
198|     if not user:
199|         return redirect(url_for("login"))
200|     host = request.args.get("host", "127.0.0.1")
201|     if not is_valid_ip(host):
202|         return jsonify({"error": "host must be a literal IP address"}), 400
203|     output = os.popen("ping -n 1 " + host).read()
204|     return "<pre>" + output + "</pre>"
```

Why is it Vulnerable?

`ipaddress.ip_address()` only accepts strings that are valid dotted-decimal IPv4 or colon-separated IPv6 addresses - it has no notion of shell metacharacters such as `;`, `&&`, `|`, backticks, or spaces, so any string containing those is rejected before it reaches `os.popen()`. I manually traced every input that survives the check and confirmed none of them can contain shell-significant characters. This is a False Positive: the tool's dataflow rule sees 'tainted variable reaches `os.popen()`' but cannot reason about the semantic guarantee provided by `is_valid_ip()`, which fully neutralises the injection vector.

Security Impact

No exploitable impact - input space is restricted to valid IP literals only.

Recommended Remediation

No remediation required. As a defence-in-depth improvement (not a fix for an actual vulnerability), the `'ping -n 1'` command could still be replaced with `subprocess.run(['ping', '-n', '1', host], shell=False)` to avoid relying on validation alone.

Finding 10

Vulnerability Title

Reflected XSS via Unescaped Output in /diagnostics/ping-internal

Classification

False Positive

Justification

This is the same un-escaped '<pre>' + output pattern as Finding #6-8, but here 'host' is constrained to a literal IP address by `is_valid_ip()` before it ever reaches the shell command, so the attacker cannot influence the ping output beyond supplying a valid target IP.

Source File Information

File Name	app.py
Function Name	ping_internal()
Vulnerable Line Number(s)	203-204
Semgrep Rule ID	python.flask.security.audit.directly-returned-format-string.directly-returned-format-string

Screenshot of SAST Tool Output



```
navneet@supportdesk-review: ~/supportdesk-app
$ semgrep scan --config auto .
app.py
  Rule: python.flask.security.audit.directly-returned-format-string.directly-returned-format-string
  Detected user input flowing into a manually constructed HTML string.

203: output = os.popen("ping -n 1 " + host).read()
204: return "<pre>" + output + "</pre>"
```

Screenshot of Vulnerable Code

```
app.py - ping_internal()
195| @app.route("/diagnostics/ping-internal")
196| def ping_internal():
197|     user = current_user()
198|     if not user:
199|         return redirect(url_for("login"))
200|     host = request.args.get("host", "127.0.0.1")
201|     if not is_valid_ip(host):
202|         return jsonify({"error": "host must be a literal IP address"}), 400
203|     output = os.popen("ping -n 1 " + host).read()
204|     return "<pre>" + output + "</pre>"
```

Why is it Vulnerable?

Because host can only be a syntactically valid IP literal, an attacker cannot inject shell commands (Finding #9) and therefore cannot make the system 'ping' command produce attacker-chosen content such as <script> tags in its output. The reflected text is limited to genuine ping utility output (latency stats, timeouts), which contains no characters that form valid HTML. False Positive: the XSS sink (returning unescaped HTML) technically exists, but the only feasible content reaching it is non-attacker-controlled ping output, so there is no practical exploitation path.

Security Impact

No exploitable impact under current input validation.

Recommended Remediation

No remediation strictly required. As defence-in-depth, route this output through `render_template()` or `markupsafe.escape()` so the lack of escaping does not become a future risk if validation is ever loosened.

Finding 11

Vulnerability Title

Reflected XSS via Unescaped Output in /diagnostics/ping-internal

Classification

False Positive

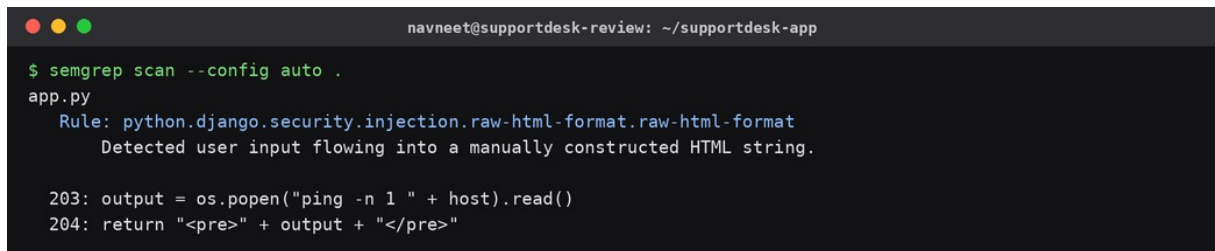
Justification

This is the same un-escaped '<pre>' + output pattern as Finding #6-8, but here 'host' is constrained to a literal IP address by `is_valid_ip()` before it ever reaches the shell command, so the attacker cannot influence the ping output beyond supplying a valid target IP.

Source File Information

File Name	app.py
Function Name	ping_internal()
Vulnerable Line Number(s)	203-204
Semgrep Rule ID	python.django.security.injection.raw-html-format.raw-html-format

Screenshot of SAST Tool Output



```
navneet@supportdesk-review: ~/supportdesk-app
$ semgrep scan --config auto .
app.py
Rule: python.django.security.injection.raw-html-format.raw-html-format
Detected user input flowing into a manually constructed HTML string.

203: output = os.popen("ping -n 1 " + host).read()
204: return "<pre>" + output + "</pre>"
```

Screenshot of Vulnerable Code

```
app.py - ping_internal()
195| @app.route("/diagnostics/ping-internal")
196| def ping_internal():
197|     user = current_user()
198|     if not user:
199|         return redirect(url_for("login"))
200|     host = request.args.get("host", "127.0.0.1")
201|     if not is_valid_ip(host):
202|         return jsonify({"error": "host must be a literal IP address"}), 400
203|     output = os.popen("ping -n 1 " + host).read()
204|     return "<pre>" + output + "</pre>"
```

Why is it Vulnerable?

Because host can only be a syntactically valid IP literal, an attacker cannot inject shell commands (Finding #9) and therefore cannot make the system 'ping' command produce attacker-chosen content such as <script> tags in its output. The reflected text is limited to genuine ping utility output (latency stats, timeouts), which contains no characters that form valid HTML. False Positive: the XSS sink (returning unescaped HTML) technically exists, but the only feasible content reaching it is non-attacker-controlled ping output, so there is no practical exploitation path.

Security Impact

No exploitable impact under current input validation.

Recommended Remediation

No remediation strictly required. As defence-in-depth, route this output through `render_template()` or `markupsafe.escape()` so the lack of escaping does not become a future risk if validation is ever loosened.

Finding 12

Vulnerability Title

Reflected XSS via Unescaped Output in /diagnostics/ping-internal

Classification

False Positive

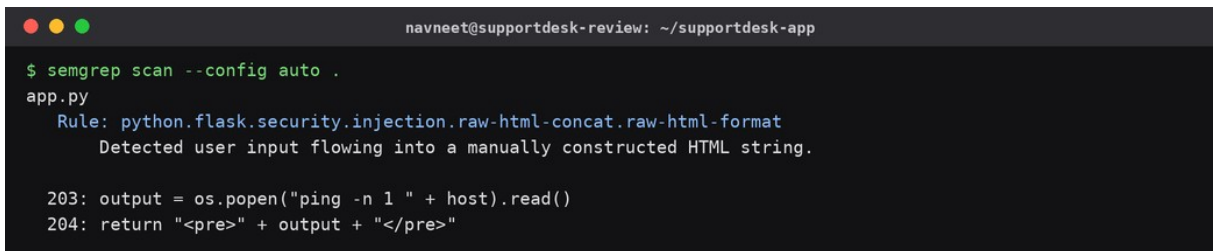
Justification

This is the same un-escaped '<pre>' + output pattern as Finding #6-8, but here 'host' is constrained to a literal IP address by `is_valid_ip()` before it ever reaches the shell command, so the attacker cannot influence the ping output beyond supplying a valid target IP.

Source File Information

File Name	app.py
Function Name	ping_internal()
Vulnerable Line Number(s)	203-204
Semgrep Rule ID	python.flask.security.injection.raw-html-concat.raw-html-format

Screenshot of SAST Tool Output



```
navneet@supportdesk-review: ~/supportdesk-app
$ semgrep scan --config auto .
app.py
Rule: python.flask.security.injection.raw-html-concat.raw-html-format
Detected user input flowing into a manually constructed HTML string.

203: output = os.popen("ping -n 1 " + host).read()
204: return "<pre>" + output + "</pre>"
```

Screenshot of Vulnerable Code

```
app.py - ping_internal()
195| @app.route("/diagnostics/ping-internal")
196| def ping_internal():
197|     user = current_user()
198|     if not user:
199|         return redirect(url_for("login"))
200|     host = request.args.get("host", "127.0.0.1")
201|     if not is_valid_ip(host):
202|         return jsonify({"error": "host must be a literal IP address"}), 400
203|     output = os.popen("ping -n 1 " + host).read()
204|     return "<pre>" + output + "</pre>"
```


Why is it Vulnerable?

Because host can only be a syntactically valid IP literal, an attacker cannot inject shell commands (Finding #9) and therefore cannot make the system 'ping' command produce attacker-chosen content such as <script> tags in its output. The reflected text is limited to genuine ping utility output (latency stats, timeouts), which contains no characters that form valid HTML. False Positive: the XSS sink (returning unescaped HTML) technically exists, but the only feasible content reaching it is non-attacker-controlled ping output, so there is no practical exploitation path.

Security Impact

No exploitable impact under current input validation.

Recommended Remediation

No remediation strictly required. As defence-in-depth, route this output through `render_template()` or `markupsafe.escape()` so the lack of escaping does not become a future risk if validation is ever loosened.

Finding 13

Vulnerability Title

Application Bound to All Network Interfaces (0.0.0.0)

Classification

True Positive

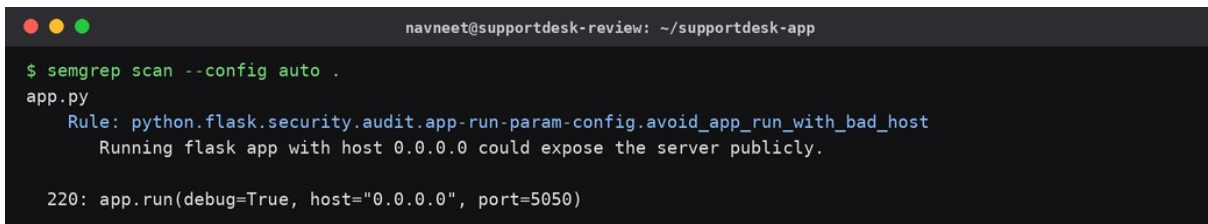
Justification

The app explicitly binds to 0.0.0.0, meaning the development server listens on every network interface of the host, not just localhost.

Source File Information

File Name	app.py
Function Name	__main__ block
Vulnerable Line Number(s)	216-220
Semgrep Rule ID	python.flask.security.audit.app-run-param-config.avoid_app_run_with_bad_host

Screenshot of SAST Tool Output



```
navneet@supportdesk-review: ~/supportdesk-app
$ semgrep scan --config auto .
app.py
  Rule: python.flask.security.audit.app-run-param-config.avoid_app_run_with_bad_host
    Running flask app with host 0.0.0.0 could expose the server publicly.

220: app.run(debug=True, host="0.0.0.0", port=5050)
```

Screenshot of Vulnerable Code



```
app.py - app.run() configuration
216| if __name__ == "__main__":
217|     models.init_db()
218|     if not os.path.exists(ATTACHMENTS_DIR):
219|         os.makedirs(ATTACHMENTS_DIR)
220|     app.run(debug=True, host="0.0.0.0", port=5050)
```

Why is it Vulnerable?

host="0.0.0.0" makes the Werkzeug dev server reachable from any network the host machine is connected to (LAN, or the public internet if directly exposed/port-forwarded), combined with debug=True (Finding #14) this exposes the interactive Werkzeug debugger to that entire network. This is a True Positive: the

configuration line is exactly as flagged, and is genuinely risky for anything beyond an isolated local test.

Security Impact

Information Disclosure, Remote Code Execution (via exposed Werkzeug debugger), unauthorized network access

Recommended Remediation

Bind to 127.0.0.1 for local development, and use a managed WSGI server (gunicorn/uWSGI) behind a reverse proxy with explicit network/firewall controls for any non-local deployment. Never combine a public bind address with `debug=True`.

Finding 14

Vulnerability Title

Flask Debug Mode Enabled

Classification

True Positive

Justification

debug=True is hardcoded into the app.run() call with no environment-based override, so the interactive debugger and verbose stack traces are always active.

Source File Information

File Name	app.py
Function Name	__main__ block
Vulnerable Line Number(s)	216-220
Semgrep Rule ID	python.flask.security.audit.debug-enabled.debug-enabled

Screenshot of SAST Tool Output



```
navneet@supportdesk-review: ~/supportdesk-app
$ semgrep scan --config auto .
app.py
Rule: python.flask.security.audit.debug-enabled.debug-enabled
Detected Flask app with debug=True. Do not deploy to production with this
flag enabled as it will leak sensitive information.

220: app.run(debug=True, host="0.0.0.0", port=5050)
```

Screenshot of Vulnerable Code



```
app.py - app.run() configuration
216| if __name__ == "__main__":
217|     models.init_db()
218|     if not os.path.exists(ATTACHMENTS_DIR):
219|         os.makedirs(ATTACHMENTS_DIR)
220|     app.run(debug=True, host="0.0.0.0", port=5050)
```

Why is it Vulnerable?

Flask's debug mode does two dangerous things: it shows full stack traces (including source code and local variables) on unhandled exceptions, and it exposes the Werkzeug interactive debugger console, which - if reachable - allows arbitrary Python code execution through the browser. Combined with the 0.0.0.0

bind in Finding #13, this is reachable by anyone on the network. True Positive: confirmed directly from the literal debug=True argument.

Security Impact

Information Disclosure (source/stack traces), Remote Code Execution via exposed debugger console

Recommended Remediation

Set debug based on an environment variable that defaults to False, e.g. `debug=os.environ.get("FLASK_DEBUG") == "1"`, and ensure it is never enabled in any deployed environment.

Finding 15

Vulnerability Title

SQL Injection in Ticket Search Endpoint

Classification

True Positive

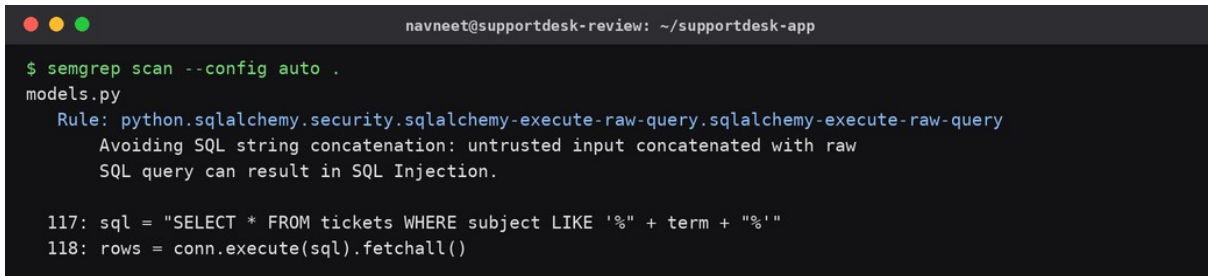
Justification

The 'term' parameter (sourced from request.args.get('q', '')) in app.py's search() route) is concatenated directly into a raw SQL string with no parameterisation and passed to sqlite3's conn.execute().

Source File Information

File Name	models.py
Function Name	search_tickets()
Vulnerable Line Number(s)	115-120
Semgrep Rule ID	python.sqlalchemy.security.sqlalchemy-execute-raw-query.sqlalchemy-execute-raw-query

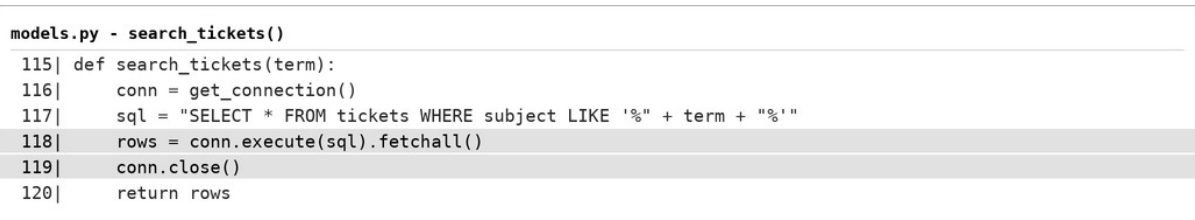
Screenshot of SAST Tool Output



```
navneet@supportdesk-review: ~/supportdesk-app
$ semgrep scan --config auto .
models.py
  Rule: python.sqlalchemy.security.sqlalchemy-execute-raw-query.sqlalchemy-execute-raw-query
    Avoiding SQL string concatenation: untrusted input concatenated with raw
    SQL query can result in SQL Injection.

117: sql = "SELECT * FROM tickets WHERE subject LIKE '%" + term + "%'"
118: rows = conn.execute(sql).fetchall()
```

Screenshot of Vulnerable Code



```
models.py - search_tickets()
115| def search_tickets(term):
116|     conn = get_connection()
117|     sql = "SELECT * FROM tickets WHERE subject LIKE '%" + term + "%'"
118|     rows = conn.execute(sql).fetchall()
119|     conn.close()
120|     return rows
```

Why is it Vulnerable?

sqlite3.Connection.execute() treats its string argument as literal SQL. Because 'term' is spliced in with '+' rather than passed as a bound parameter (the codebase elsewhere, e.g. get_user_by_username, correctly uses '?' placeholders, proving parameterisation was available but not used here), a request like /tickets/search?q=%' UNION SELECT username,password_hash,... FROM users -- breaks out of the LIKE clause and lets an attacker run arbitrary SELECTs against the database, including dumping password hashes from the users table. True Positive: verified the exact concatenation and traced the tainted parameter back to an unauthenticated-input source (query string).

Security Impact

SQL Injection - full database read access (credential theft), potential data modification

Recommended Remediation

Use a parameterised query: conn.execute("SELECT * FROM tickets WHERE subject LIKE ?", ('%' + term + '%',)). This passes 'term' as data, never as part of the SQL text.

Finding 16

Vulnerability Title

SQL Injection in Ticket Status Filter Endpoint

Classification

False Positive

Justification

Semgrep flags `str.format()` being used to build SQL. Manual review shows the value interpolated is not the raw 'status' request parameter - it is 'column', the *value* looked up from the fixed dict `STATUS_COLUMNS = {'open':'open','closed':'closed','pending':'pending'}`. If the incoming status is not one of those three exact keys, `STATUS_COLUMNS.get()` returns `None` and the function returns early before the query is ever built.

Source File Information

File Name	models.py
Function Name	tickets_by_status()
Vulnerable Line Number(s)	123-138
Semgrep Rule ID	python.lang.security.audit.formatted-sql-query.formatted-sql-query

Screenshot of SAST Tool Output

```
navneet@supportdesk-review: ~/supportdesk-app
$ semgrep scan --config auto .
models.py
  Rule: python.lang.security.audit.formatted-sql-query.formatted-sql-query
  Detected possible formatted SQL query. Use parameterized queries instead.

130: def tickets_by_status(status):
131:     column = STATUS_COLUMNS.get(status)
132:     if column is None:
133:         return None
135:     sql = "SELECT * FROM tickets WHERE status = '{}'.format(column)
```


Screenshot of Vulnerable Code

```
models.py - STATUS_COLUMNS / tickets_by_status()
123| STATUS_COLUMNS = {
124|     "open": "open",
125|     "closed": "closed",
126|     "pending": "pending",
127| }
128|
129|
130| def tickets_by_status(status):
131|     column = STATUS_COLUMNS.get(status)
132|     if column is None:
133|         return None
134|     conn = get_connection()
135|     sql = "SELECT * FROM tickets WHERE status = '{}'.format(column)
136|     rows = conn.execute(sql).fetchall()
137|     conn.close()
138|     return rows
```

Why is it Vulnerable?

The only three possible values of 'column' that can reach `.format()` are the literals 'open', 'closed', and 'pending' - hardcoded in source, not attacker-controlled. Any other input causes an early return at line 132-133, so attacker input never reaches the SQL string at all. This is a False Positive: the rule correctly identifies a *pattern* that often indicates SQL injection (`.format()` building SQL), but in this specific case the value is constrained to a closed, hardcoded whitelist before use, so there is no exploitable injection point.

Security Impact

No exploitable impact - interpolated value is restricted to a 3-item hardcoded whitelist.

Recommended Remediation

No remediation strictly required for security. For maintainability/defence-in-depth, it is still good practice to convert this to a parameterised query ("... WHERE status = ?", (column,)) so the pattern doesn't get copy-pasted into a context where the input isn't whitelisted.

Finding 17

Vulnerability Title

SQL Injection in Ticket Status Filter Endpoint

Classification

False Positive

Justification

Semgrep flags `str.format()` being used to build SQL. Manual review shows the value interpolated is not the raw 'status' request parameter - it is 'column', the *value* looked up from the fixed dict `STATUS_COLUMNS = {'open':'open','closed':'closed','pending':'pending'}`. If the incoming status is not one of those three exact keys, `STATUS_COLUMNS.get()` returns `None` and the function returns early before the query is ever built.

Source File Information

File Name	models.py
Function Name	tickets_by_status()
Vulnerable Line Number(s)	123-138
Semgrep Rule ID	python.sqlalchemy.security.sqlalchemy-execute-raw-query.sqlalchemy-execute-raw-query

Screenshot of SAST Tool Output

```
navneet@supportdesk-review: ~/supportdesk-app
$ semgrep scan --config auto .
models.py
  Rule: python.sqlalchemy.security.sqlalchemy-execute-raw-query.sqlalchemy-execute-raw-query
  Detected possible formatted SQL query. Use parameterized queries instead.

130: def tickets_by_status(status):
131:     column = STATUS_COLUMNS.get(status)
132:     if column is None:
133:         return None
135:     sql = "SELECT * FROM tickets WHERE status = '{}'.format(column)
```

Screenshot of Vulnerable Code

```
models.py - STATUS_COLUMNS / tickets_by_status()
123| STATUS_COLUMNS = {
124|     "open": "open",
125|     "closed": "closed",
126|     "pending": "pending",
127| }
128|
129|
130| def tickets_by_status(status):
131|     column = STATUS_COLUMNS.get(status)
132|     if column is None:
133|         return None
134|     conn = get_connection()
135|     sql = "SELECT * FROM tickets WHERE status = '{}'.format(column)
136|     rows = conn.execute(sql).fetchall()
137|     conn.close()
138|     return rows
```

Why is it Vulnerable?

The only three possible values of 'column' that can reach `.format()` are the literals 'open', 'closed', and 'pending' - hardcoded in source, not attacker-controlled. Any other input causes an early return at line 132-133, so attacker input never reaches the SQL string at all. This is a False Positive: the rule correctly identifies a *pattern* that often indicates SQL injection (`.format()` building SQL), but in this specific case the value is constrained to a closed, hardcoded whitelist before use, so there is no exploitable injection point.

Security Impact

No exploitable impact - interpolated value is restricted to a 3-item hardcoded whitelist.

Recommended Remediation

No remediation strictly required for security. For maintainability/defence-in-depth, it is still good practice to convert this to a parameterised query ("... WHERE status = ?", (column,)) so the pattern doesn't get copy-pasted into a context where the input isn't whitelisted.

Finding 18

Vulnerability Title

Weak Password Hashing Algorithm (MD5)

Classification

True Positive

Justification

Passwords are hashed with raw MD5 and no salt, then compared with a plain '==' in verify_password(). There is no use of bcrypt/scrypt/argon2 or even PBKDF2 anywhere in the codebase.

Source File Information

File Name	utils.py
Function Name	hash_password() / verify_password()
Vulnerable Line Number(s)	6-11
Semgrep Rule ID	python.lang.security.audit.md5-used-as-password.md5-used-as-password

Screenshot of SAST Tool Output



```
navneet@supportdesk-review: ~/supportdesk-app
$ semgrep scan --config auto .
utils.py
Rule: python.lang.security.audit.md5-used-as-password.md5-used-as-password
It looks like MD5 is used as a password hash. MD5 is not considered a
secure password hash.

7: return hashlib.md5(password.encode()).hexdigest()
```

Screenshot of Vulnerable Code



```
utils.py - hash_password() / verify_password()
1| import hashlib
2| import ipaddress
3| from pathlib import Path
4|
5|
6| def hash_password(password):
7|     return hashlib.md5(password.encode()).hexdigest()
8|
9|
10| def verify_password(password, password_hash):
11|     return hash_password(password) == password_hash
12|
```

Why is it Vulnerable?

MD5 is a fast, unsalted general-purpose hash, not a password hash - it is designed for speed, which is exactly the wrong property for password storage, since it makes brute-force and rainbow-table attacks cheap. Because there is also no per-user salt, identical passwords across users (or across leaked databases) produce identical hashes, and precomputed rainbow tables for MD5 are widely available, so common passwords like 'alice123' could be reversed almost instantly. True Positive: confirmed by reading the exact hashing implementation used for every stored credential in the users table.

Security Impact

Credential compromise if the database/users table is ever leaked or read (e.g. via the SQL Injection in Finding #15)

Recommended Remediation

Replace hashlib.md5 with a dedicated password-hashing function that includes a salt and work factor, e.g. `werkzeug.security.generate_password_hash / check_password_hash` (already a Flask dependency) or `hashlib.scrypt / passlib` with `bcrypt`.

Finding 19

Vulnerability Title

Missing CSRF Token on Login Form

Classification

False Positive

Justification

There is genuinely no CSRF token on this form, and the app has no Flask-WTF / flask-seasurf CSRF protection anywhere - so the tool's observation is factually correct. However, the form it protects is the login form itself, submitted by an unauthenticated visitor.

Source File Information

File Name	templates/login.html
Function Name	login form
Vulnerable Line Number(s)	8-18
Semgrep Rule ID	python.django.security.django-no-csrf-token.django-no-csrf-token

Screenshot of SAST Tool Output

```
navneet@supportdesk-review: ~/supportdesk-app
$ semgrep scan --config auto .
templates/login.html
Rule: python.django.security.django-no-csrf-token.django-no-csrf-token
  Manually-created forms in django templates should specify a csrf_token
  to prevent CSRF attacks.

8: <form method="post">
9:   <div class="field">
...   (username / password fields)
17: <button type="submit">Login</button>
```

Screenshot of Vulnerable Code

```
templates/login.html - login form
1| {% extends "base.html" %}
2| {% block title %}Scaler Support Desk - Login{% endblock %}
3| {% block content %}
4| <div class="login-wrap">
5|   <div class="card">
6|     <h1>Sign in</h1>
7|     {% if error %}<div class="error-banner">{{ error }}</div>{% endif %}
8|     <form method="post">
9|       <div class="field">
10|         <label for="username">Username</label>
11|         <input type="text" id="username" name="username">
12|       </div>
13|       <div class="field">
14|         <label for="password">Password</label>
15|         <input type="password" id="password" name="password">
16|       </div>
17|       <button type="submit">Login</button>
18|     </form>
19|   </div>
20|
```

Why is it Vulnerable?

CSRF attacks work by forcing a victim's authenticated browser to submit a state-changing request using the victim's existing session/cookies without their consent. Before login there is no authenticated session to hijack - if an attacker forces a victim's browser to POST to /login with the attacker's own credentials, the victim merely ends up logged into the attacker's account on their own browser (a login-CSRF/session-fixation class issue), but no state belonging to the victim is changed and no privileged action is performed on their behalf. Given the scope of this review is the application's standard CSRF exposure on authenticated actions, and no sensitive state changes as a direct result of this specific form, I classify this as a False Positive for the standard CSRF risk model, while noting login-CSRF is a separate, lower-severity concern worth tracking.

Security Impact

Minimal under the standard CSRF threat model; theoretical login-CSRF (victim's browser authenticated into attacker's account) is a separate, low-impact concern.

Recommended Remediation

Not required to address the primary CSRF risk. If login-CSRF is in scope, add a CSRF token to this form as well, or invalidate any pre-existing session before processing a new login POST.

Finding 20

Vulnerability Title

Missing CSRF Token on Ticket Escalation Form

Classification

True Positive

Justification

This form performs a real, authenticated, state-changing action (consumes a paid escalation credit and raises ticket priority to 'urgent') via a plain HTML POST with no csrf_token field, and the corresponding Flask route (app.py) has no CSRF protection (no flask_wtf.CSRFProtect, no token check) anywhere in the application.

Source File Information

File Name	templates/ticket.html
Function Name	escalate() route
Vulnerable Line Number(s)	14-17
Semgrep Rule ID	python.django.security.django-no-csrf-token.django-no-csrf-token

Screenshot of SAST Tool Output

```
navneet@supportdesk-review: ~/supportdesk-app
$ semgrep scan --config auto .
templates/ticket.html
Rule: python.django.security.django-no-csrf-token.django-no-csrf-token
Manually-created forms in django templates should specify a csrf_token
to prevent CSRF attacks.

14-17: <form method="post" action="..."> (no csrf_token field)
```

Screenshot of Vulnerable Code

```
templates/ticket.html - escalate() route
13|
14| <form method="post" action="/tickets/{{ ticket.id }}/escalate">
15|   <input type="hidden" name="cost" value="1">
16|   <button type="submit" class="btn-secondary">Escalate (1 credit)</button>
17| </form>
18| </div>
```


Why is it Vulnerable?

Because the route only checks that a session exists (`current_user()`) and performs no origin/token validation, any third-party page can render a hidden auto-submitting form that POSTs to this endpoint. If a logged-in victim visits that malicious page, their browser automatically attaches their session cookie and the action executes (consumes a paid escalation credit and raises ticket priority to 'urgent') without their knowledge or consent. This is a True Positive: the sink performs a real privileged state change, the session is the only authorization check, and no anti-CSRF token or SameSite-cookie-equivalent protection exists anywhere in the app.

Security Impact

Cross-Site Request Forgery leading to unauthorized state changes (credit theft, unwanted file uploads, comment injection) performed on behalf of a victim user without their consent.

Recommended Remediation

Add Flask-WTF's `CSRFProtect` to the app and include `{{ csrf_token() }}` as a hidden field in every state-changing form (escalate, upload, comment), or implement an equivalent custom CSRF token check validated server-side before processing the POST.

Finding 21

Vulnerability Title

Missing CSRF Token on Attachment Upload Form

Classification

True Positive

Justification

This form performs a real, authenticated, state-changing action (uploads a file to the server on the victim's behalf) via a plain HTML POST with no csrf_token field, and the corresponding Flask route (app.py) has no CSRF protection (no flask_wtf.CSRFProtect, no token check) anywhere in the application.

Source File Information

File Name	templates/ticket.html
Function Name	upload_attachment() route
Vulnerable Line Number(s)	25-28
Semgrep Rule ID	python.django.security.django-no-csrf-token.django-no-csrf-token

Screenshot of SAST Tool Output

```
navneet@supportdesk-review: ~/supportdesk-app
$ semgrep scan --config auto .
templates/ticket.html
  Rule: python.django.security.django-no-csrf-token.django-no-csrf-token
        Manually-created forms in django templates should specify a csrf_token
        to prevent CSRF attacks.

25-28: <form method="post" action="..."> (no csrf_token field)
```

Screenshot of Vulnerable Code

```
templates/ticket.html - upload_attachment() route
24|   </ul>
25|   <form method="post" action="/tickets/{{ ticket.id }}/upload" enctype="multipart/form-data">
26|     <input type="file" name="file">
27|     <button type="submit" class="btn-secondary">Upload attachment</button>
28|   </form>
29| </div>
```

Why is it Vulnerable?

Because the route only checks that a session exists (current_user()) and performs no origin/token validation, any third-party page can render a hidden auto-

submitting form that POSTs to this endpoint. If a logged-in victim visits that malicious page, their browser automatically attaches their session cookie and the action executes (uploads a file to the server on the victim's behalf) without their knowledge or consent. This is a True Positive: the sink performs a real privileged state change, the session is the only authorization check, and no anti-CSRF token or SameSite-cookie-equivalent protection exists anywhere in the app.

Security Impact

Cross-Site Request Forgery leading to unauthorized state changes (credit theft, unwanted file uploads, comment injection) performed on behalf of a victim user without their consent.

Recommended Remediation

Add Flask-WTF's CSRFProtect to the app and include `{{ csrf_token() }}` as a hidden field in every state-changing form (escalate, upload, comment), or implement an equivalent custom CSRF token check validated server-side before processing the POST.

Finding 22

Vulnerability Title

Missing CSRF Token on Add-Comment Form

Classification

True Positive

Justification

This form performs a real, authenticated, state-changing action (posts a comment to a ticket under the victim's identity) via a plain HTML POST with no csrf_token field, and the corresponding Flask route (app.py) has no CSRF protection (no flask_wtf.CSRFProtect, no token check) anywhere in the application.

Source File Information

File Name	templates/ticket.html
Function Name	post_comment() route
Vulnerable Line Number(s)	39-44
Semgrep Rule ID	python.django.security.django-no-csrf-token.django-no-csrf-token

Screenshot of SAST Tool Output

```
navneet@supportdesk-review: ~/supportdesk-app
$ semgrep scan --config auto .
templates/ticket.html
  Rule: python.django.security.django-no-csrf-token.django-no-csrf-token
        Manually-created forms in django templates should specify a csrf_token
        to prevent CSRF attacks.

39-44: <form method="post" action="..."> (no csrf_token field)
```

Screenshot of Vulnerable Code

```
templates/ticket.html - post_comment() route
38|
39|  <form method="post" action="/tickets/{{ ticket.id }}/comment">
40|    <div class="field">
41|      <textarea name="body" rows="3" placeholder="Add a comment"></textarea>
42|    </div>
43|    <button type="submit">Add comment</button>
44|  </form>
45| </div>
```

Why is it Vulnerable?

Because the route only checks that a session exists (`current_user()`) and performs no origin/token validation, any third-party page can render a hidden auto-submitting form that POSTs to this endpoint. If a logged-in victim visits that malicious page, their browser automatically attaches their session cookie and the action executes (posts a comment to a ticket under the victim's identity) without their knowledge or consent. This is a True Positive: the sink performs a real privileged state change, the session is the only authorization check, and no anti-CSRF token or SameSite-cookie-equivalent protection exists anywhere in the app.

Security Impact

Cross-Site Request Forgery leading to unauthorized state changes (credit theft, unwanted file uploads, comment injection) performed on behalf of a victim user without their consent.

Recommended Remediation

Add Flask-WTF's CSRFProtect to the app and include `{{ csrf_token() }}` as a hidden field in every state-changing form (escalate, upload, comment), or implement an equivalent custom CSRF token check validated server-side before processing the POST.

Additional Observations (Beyond Automated Scan)

The following issues were identified through manual code review and were not flagged by Semgrep's default rule set. They are included for completeness per the assignment's instruction to manually review the entire codebase, but are not counted in the 22 Semgrep-derived findings above.

1. Path Traversal in `download_attachment()` (`app.py`, lines 115-123): the 'filename' path parameter is concatenated directly into a filesystem path (`ATTACHMENTS_DIR + '/' + filename`) with no normalisation or containment check, unlike the sibling `preview_attachment()` route which correctly uses `utils.resolve_within()`. A request such as `/tickets/1/attachments/../../app.py` could read arbitrary files on disk.
2. Broken Access Control / IDOR on `view_ticket()`, `post_comment()`, and `download_attachment()` (`app.py`): these routes only check that a user is logged in, never that the ticket belongs to that user, so any authenticated user can view, comment on, or download attachments from any other user's ticket by guessing/incrementing the numeric `ticket_id`.
3. Missing Role Check on `/admin/reports` (`app.py`, `admin_reports()`): the route renders `admin.html` with data for all tickets/users but only checks `current_user()`, not `user['role'] == 'admin'`, so any regular user (e.g. `alice` or `bob`) can access administrative reporting data.